AI-Assisted Multi-Format Academic Content Generation: A Python and Streamlit Case Study on the Internet of Things

Design, Implementation and Evaluation of a Source-Grounded Content Studio

TEGUIA FRANK • UID 26MCA20153 • 26CAP-608 • Chandigarh University • October 2026

Abstract

Long technical documents contain the evidence that students and researchers need, yet the same evidence rarely survives the jump between prose, slides, posters and assessable reports. Manual re-authoring is slow, duplicates effort and introduces inconsistencies in facts, terminology and citation. This report presents an offline, deterministic multi-format content generation system that converts a 92-page Internet of Things (IoT) white book into four coordinated academic artefacts: a structured study guide, a ten-slide presentation deck, an A4 infographic and a refined scholarly text. The system combines PDF text extraction, TF-IDF sentence ranking with a maximal marginal relevance penalty, curated domain scaffolding, python-pptx layout generation, Pillow poster rendering and a rule-based academic-style refiner with a six-criterion audit. A Streamlit interface exposes the complete pipeline and packages eight files plus a JSON manifest. On the demonstration corpus, the guide compresses 24,972 source words to 2,216 words (8.9%), retains four topics, twelve glossary terms, eleven research fields and twelve examination questions, and the complete run takes 21.31 s. The audit verifies five of six quality criteria (83.3%) and explicitly flags readability as only partially verified. The contribution is not a replacement for scholarly judgement but a transparent, reproducible production layer that keeps every published claim traceable to the source or to a reviewed domain fact.

Keywords: Internet of Things; multi-format generation; extractive summarisation; Streamlit; academic writing; reproducibility.

1. Introduction

1.1 The Multi-Format Problem

Academic communication is multimodal. A reader first meets a topic as a narrative, revises it through structured notes, presents it as a sequence of claims, and remembers it through a visual abstraction. Yet the same evidence is usually rewritten by hand for each format. This duplication is expensive and error-prone: a statistic can drift between the abstract and a slide, a definition can be shortened until it becomes incorrect, and a reference can disappear during reformatting. The problem is not merely clerical. Each format has a different grammar. Prose supports qualification and argument, slides demand compression and sequence, posters require hierarchy and visual causality, and scholarly writing requires explicit citation and a controlled vocabulary. A production system must therefore preserve one evidence base while allowing every artefact to obey its own conventions.

Multi-format generation addresses this gap by inserting a structured representation between the source and the outputs. Instead of asking a model to write four unrelated documents, the system extracts a sentence store, ranks and clusters it, and then renders the same selected evidence through format-specific engines. The approach follows the dual-coding principle that verbal and visual representations reinforce memory, while respecting cognitive-load limits: a slide is not a page, and a poster is not a paper with larger type. The engineering challenge is to keep the evidence invariant while the representation changes.

2. Background and Design Principles

2.1 The IoT Evidence Base

The Internet of Things is a demanding test case. The Inria white book used here is 92 pages and 24,972 words, and it moves between market statistics, radio protocols, operating systems, cryptographic primitives, regulation and research policy [1]. It contains 204 detected sections and a dense vocabulary of acronyms, including CPS, WoT, SUIT, 6LoWPAN, OSCORE and RIOT. A summary that merely shortens sentences will fail: it must preserve the relationship between a constraint such as microWatt energy, a design response such as lightweight cryptography, and a societal consequence such as security or digital sovereignty.

The domain also supplies verifiable anchor facts. Between 2010 and 2020 the share of IoT devices among all connected devices rose from 10% to more than 50%, and the installed base reached roughly 10 billion units [1]. More than 28 billion microcontrollers were shipped in 2018, a constrained node may have only about 0.001% of the memory budget of a conventional Internet-connected machine, and ChaCha20 can deliver roughly a 30% performance improvement over AES on microcontrollers without hardware acceleration [1], [5]. These numbers make it possible to test whether a generated artefact has retained the source's quantitative claims rather than merely its topic words.

2.2 From Summary to Format Transfer

Automatic summarisation is commonly divided into extractive and abstractive approaches. Extractive methods select existing sentences and are therefore easy to audit; abstractive methods paraphrase and can be more fluent but may introduce unsupported claims. This project deliberately chooses the extractive side for its evidence layer. Sentence selection uses TF-IDF vectors, a length preference and a maximal marginal relevance (MMR) penalty that rejects candidates too similar to those already chosen. The result is a compact, traceable set of sentences that a reader can compare with the source.

Format transfer is then a rendering problem rather than a second generation problem. The study guide exposes the full argument; the deck reduces it to ten claims with speaker notes; the poster converts causes and consequences into a visual hierarchy; and the refinement phase rewrites the abstract in a formal register with citations. Because all four artefacts read from the same structured guide, terminology and numbers remain consistent across formats. The trade-off is that the prose can sound compressed, and the system cannot resolve contradictions that are present in the source itself.

2.3 Design Principles

Five principles guided the implementation. First, source grounding: every generated sentence must originate in the extracted text or in an explicitly reviewed domain fact. Second, deterministic reproducibility: the same input and metadata must produce the same bundle, which rules out hidden sampling. Third, format-native composition: each renderer has its own grid, typography and density rules rather than exporting one document to several file types. Fourth, measurable quality: compression, readability, audit status and runtime are recorded in a manifest. Fifth, local-first privacy: no source text leaves the machine and no API key is required. Table 1 contrasts this approach with manual authoring and a single chat-based generator.

Approach	Strength	Limitation
Manual authoring	Full editorial judgement and format-specific craft	Typically 10–15 hours per document; facts drift between formats
Single chat prompt	Fast prose and layout suggestions	Opaque sourcing, variable output and a mandatory API dependency
Proposed pipeline	Traceable extraction, deterministic layout and a run manifest	Extractive prose, a curated domain profile and no semantic entailment

Table 1 — Three production approaches. The pipeline trades generative fluency for traceability and reproducibility.

3. System Architecture

3.1 Pipeline Overview

The system is organised as a linear pipeline with a shared evidence layer. A source document enters an extraction stage that produces a SourceDocument: cleaned text, page count, word count, sentence list and detected sections. Phase 1 converts that object into a StudyGuide containing topics, glossary entries, a revision matrix, facts and questions. Phases 2 and 3 render the guide into a deck and a poster, while Phase 4 produces a refined text and an audit. The orchestrator records timings, file sizes and metrics in a manifest, and the Streamlit layer exposes the same functions through a browser interface. Figure 1 shows the resulting architecture.

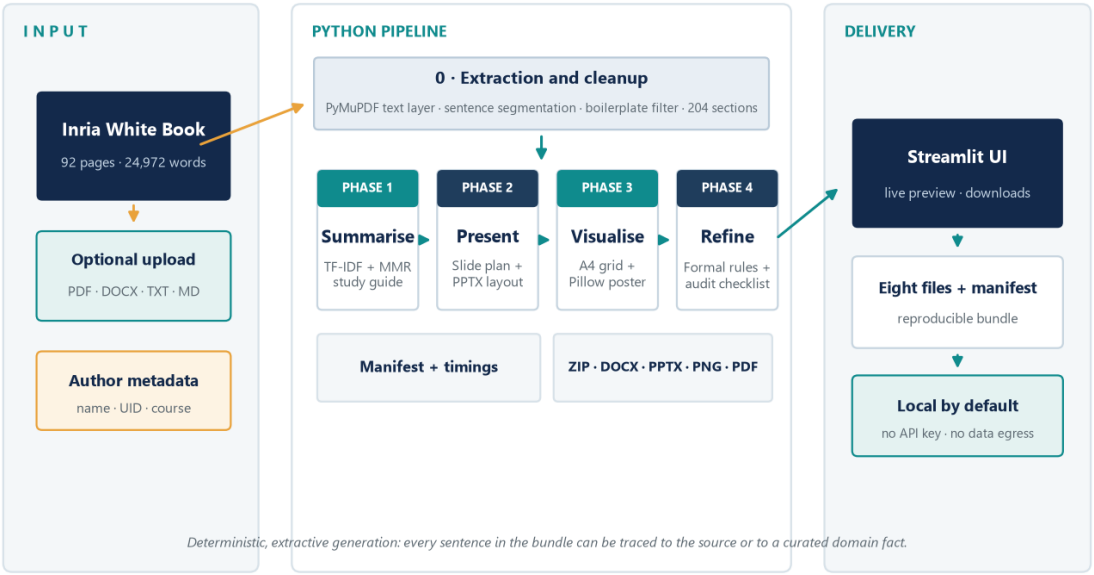


Figure 1 — System architecture. One extracted evidence base feeds four format engines; the orchestrator records metrics and packages the bundle.

3.2 Components and Responsibilities

The codebase separates ingestion, linguistic utilities, domain knowledge, phase logic, export and orchestration. This separation matters because each layer can be tested in isolation: text utilities do not know about PowerPoint, and the deck builder does not read PDF files. Table 2 summarises the modules and their contracts.

Module	Responsibility	Output
extract.py	PDF, DOCX and text ingestion; whitespace and boilerplate cleanup; page and section statistics	SourceDocument
textutils.py	Sentence segmentation, TF-IDF vectors, MMR ranking and readability	Scored sentences
domain.py	Reviewed IoT glossary, fact bank, research fields and topic queries	Domain profile
phase1_notes.py	Topic synthesis, glossary, matrix, question bank and reviser	StudyGuide
phase2_slides.py	Ten-slide narrative and 16:9 layout engine	PPTX deck
phase3_infographic.py	A4 grid, charts and cause-effect panels	PNG and PDF
phase4_refine.py	Formal-style rules, citations and six-criterion audit	RefinementResult
exporters.py	DOCX styling, Markdown and ZIP packaging	DOCX, MD, ZIP
orchestrator.py	Sequencing, timings, manifest and RunResult	Run directory

Table 2 — Module boundaries. Each phase consumes a typed object and returns a typed artefact, which keeps the pipeline testable.

3.3 Technology Stack

The implementation uses only local Python libraries, which keeps the system portable and removes the cost and privacy risks of a hosted language model. PyMuPDF provides fast, layout-aware PDF text extraction [8]; python-docx and python-pptx generate Office files without requiring Microsoft Office on the server; Pillow renders the poster at print resolution; matplotlib produces the evaluation figures; and Streamlit supplies the web interface. Table 3 lists the versions used for the reported run.

Component	Version	Role
Python	3.13.15	Runtime and standard library
Streamlit	1.66.1	Browser interface and downloads
PyMuPDF	1.28.2	PDF text extraction
python-docx	1.2.0	DOCX export and this report
python-pptx	1.0.2	Ten-slide deck generation
Pillow	12.3.0	A4 poster rendering
Matplotlib	3.11.2	Evaluation figures
pandas	3.0.6	Tabular previews in the interface

Table 3 — Technology stack. All processing is local and no API key is required.

3.4 The Streamlit Interface

The interface is designed around a single run rather than a chat. The sidebar offers a built-in 92-page source or an upload in PDF, DOCX, TXT or Markdown, exposes author, UID, course, institution and title fields, and starts the pipeline with one button. A progress bar and status panel report each phase. Once the run completes, five tabs present the study guide, deck, poster, refinement and download bundle, with inline previews and individual download buttons. Because the result is stored in Streamlit session state, a user can move between tabs without rerunning the pipeline.

3.5 Data Contracts and the Manifest

Every phase returns a dataclass rather than a loose dictionary. SourceDocument carries the cleaned text and source statistics; StudyGuide carries topics, glossary, matrix, questions, reviser and metrics; RefinementResult carries the refined abstract, glossary, audit rows and readability before and after; RunResult carries the output directory, the three objects, the file map and the timings. The orchestrator serialises the same information into manifest.json, which becomes the single source of truth for this report. That contract is what allows the evaluation in Section 5 to quote exact byte sizes, compression ratios and runtimes instead of estimates.

4. Implementation of the Four Phases

The four phases share one evidence base but use different algorithms and output formats. This section describes each phase in implementation order and reports the artefact produced by the demonstration run. The emphasis is on the decisions that connect a linguistic method to a layout constraint, because that connection is where multi-format systems usually fail.

4.1 Phase 1: Summarisation and Structuring

Extraction begins with PyMuPDF, which returns the text layer page by page. The loader normalises whitespace, removes repeated running heads, page numbers and URL fragments, and records page, word and section statistics. On the white book this yields 24,972 words, 927 sentences and 204 detected sections. Sentences are then sanitised, filtered for minimum length and deduplicated at a cosine threshold of 0.86, which removes repeated definitions that appear in both the body and the glossary of the source.

Ranking uses a compact TF-IDF vector space. Each sentence is represented by normalised term frequencies multiplied by an inverse document frequency of log((1 + N) / (1 + df)) + 1. A query vector is compared with each sentence using cosine similarity, and sentences outside the 18–48 word range receive a 0.82 length penalty. Selection follows maximal marginal relevance: a candidate is rejected when its similarity to an already selected sentence exceeds the diversity threshold of 0.32. For the IoT profile, four reviewed topic queries drive retrieval, and the resulting evidence is combined with a curated glossary and fact bank so that the published guide never contains PDF debris such as running heads or table-of-contents lines.

The phase produces a 2,216-word guide, which is 8.9% of the source. It contains 4 topics, 12 glossary terms, 11 research fields, 12 reviewed facts and 12 examination questions. Continuous prose measures 16.16 words per sentence, a Flesch reading ease of 11.1, a Flesch–Kincaid grade of 15.7 and a lexical density of 69.4%. Figure 2 contrasts the word budget with the inventory that the guide retains.

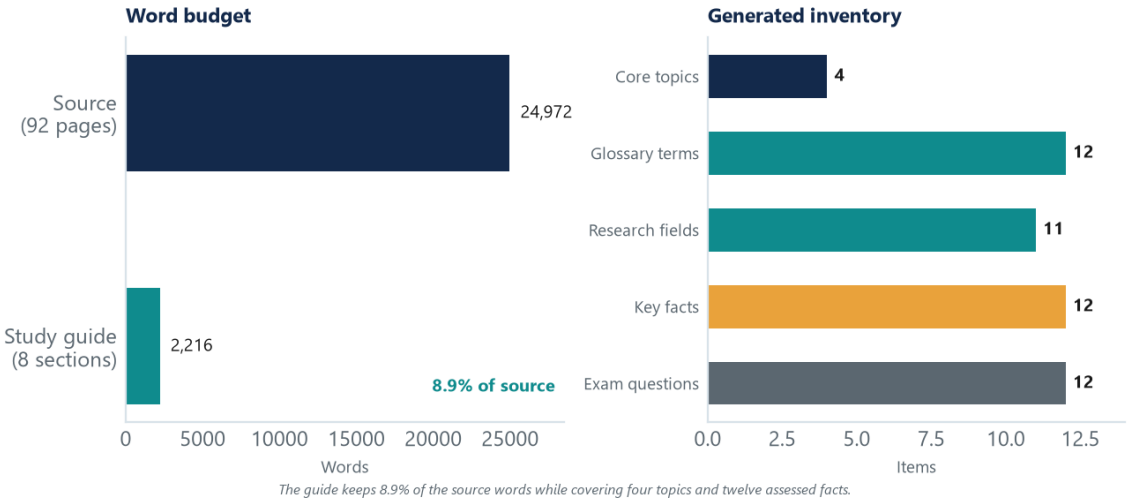


Figure 2 — Phase 1 output. The guide keeps 8.9% of the source words while retaining four topics, twelve glossary terms, eleven research fields, twelve facts and twelve questions.

4.2 Phase 2: Presentation Deck Generation

The deck is generated from a fixed ten-slide narrative rather than from a generic template. The arc moves from context and objectives, through method and comparison, to the problem, a secure-update process, a timeline,

recommendations and a conclusion. Each slide is described by a SlidePlan that carries a number, kicker, title, bullet content, layout name, visual suggestion and speaker note. The layout engine supports bullets, numbered objective cards, a process arrow, a split-screen comparison, a two-column risk map, a five-chevron flow, a horizontal timeline, solution cards and a closing panel. Table 4 summarises the sequence.

#	Kicker	Slide title	Layout and visual
01	ACADEMIC PRESENTATION	Internet of Things: Foundations and Challenges	Full-bleed navy cover
02	CONTEXT	Introduction: From Ubiquitous Computing to a Global IoT	Bullets and cloud–edge–thing diagram
03	OBJECTIVES	Objectives of the Analysis	Four numbered cards
04	METHOD	Methodology: A Four-Step Evidence Pipeline	Process arrow
05	COMPARISON	Developed vs Constrained Environments	Split-screen table
06	PROBLEM	Challenges and Their Societal Impact	Two-column risk map
07	PROCESS	Secure Firmware Update Lifecycle (SUIT)	Five-chevron flow
08	TIMELINE	Evolution of IoT and Standards	Horizontal timeline
09	SOLUTIONS	Design and Implementation Recommendations	Four solution cards
10	CONCLUSION	Conclusion: Sovereignty by Design	Navy closing panel

Table 4 — The ten-slide narrative. Every slide carries a speaker note and a visual suggestion in addition to the visible content.

The renderer uses a 13.333 × 7.5 inch canvas, a navy and teal palette, a consistent chrome with kicker and slide number, and an author footer. Speaker notes are placed in the notes pane so that the deck can be presented without a separate script. The resulting file is 63,613 bytes and opens as a standard PowerPoint presentation. Figure 3 shows the complete storyboard.

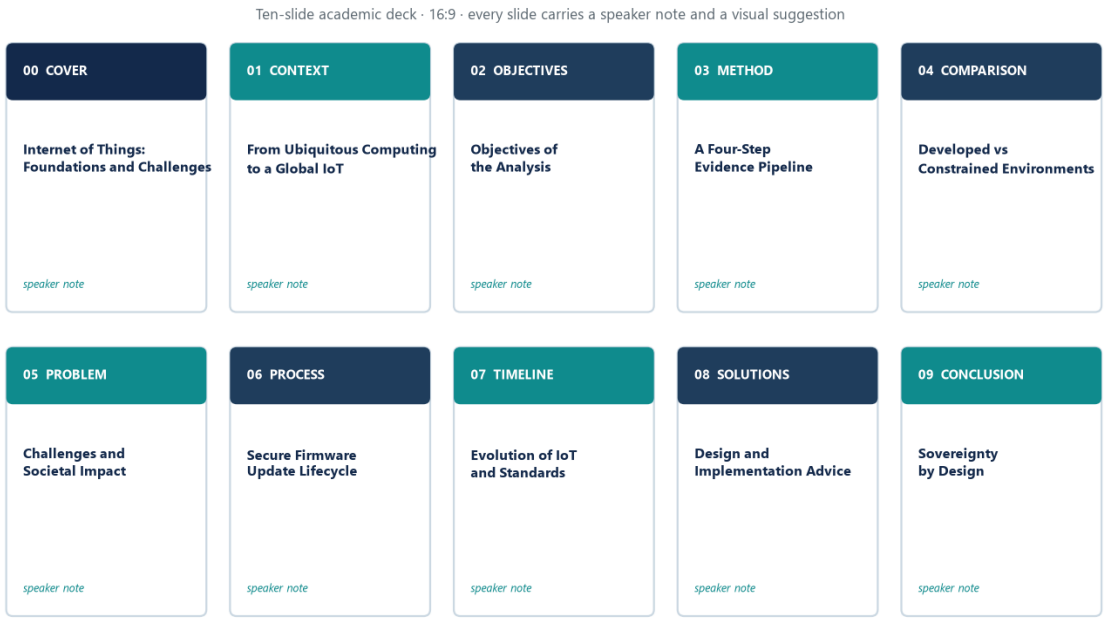


Figure 3 — Phase 2 storyboard. Ten slides form a single argument, from context to a sovereignty-by-design conclusion.

4.3 Phase 3: Visual Infographic Synthesis

The infographic is rendered directly at A4 print resolution: 1654 × 2339 pixels at 200 dpi. Pillow draws a header with the title and author metadata, a problem panel with the scale statistics, a cause-and-effect section that links technical constraints to societal outcomes, a solutions panel, a stakeholder-action strip and a footer. A custom word-wrap routine measures text width before drawing, which prevents the overflow that occurs when a fixed character count is used with proportional fonts. The renderer exports both PNG for screens and PDF for print; the PDF is generated at the same physical size so that type remains sharp.

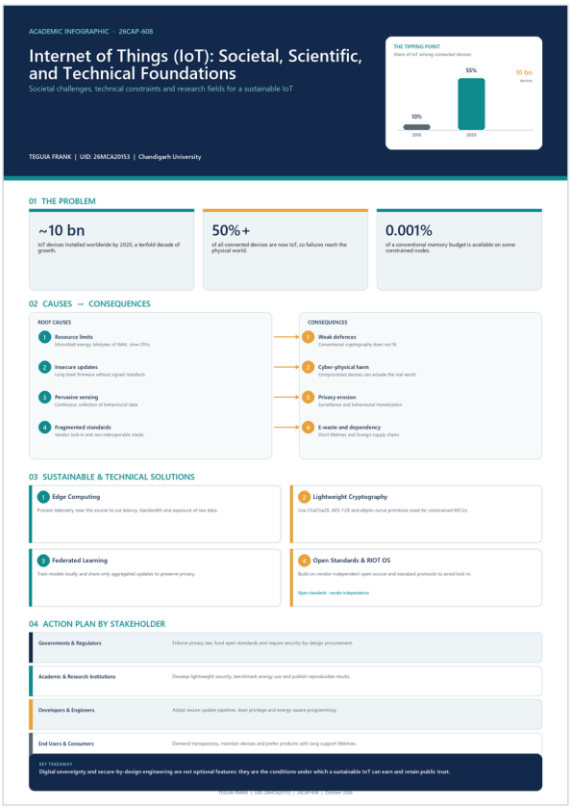


Figure 4 — Phase 3 A4 infographic. The poster converts the guide's causes and consequences into a single visual hierarchy for print or corridor display.

4.4 Phase 4: Scholarly Writing Refinement

The refinement phase is transparent by design. A rule set expands contractions, replaces informal verbs and removes filler phrases; first-person forms are converted to impersonal academic subjects. The refined abstract is then composed with six IEEE-style citations and a controlled glossary of twelve IoT terms. In the demonstration run the abstract grows from 79 to 203 words, average sentence length rises from 19.75 to 25.38 words, and Flesch reading ease falls from 20.8 to -1.9. That fall is not hidden: the formal register adds nominalisations, citations and technical terms, and the audit records readability as partially verified rather than claiming an improvement that the metric does not support.

The phase also performs a six-criterion peer-review audit covering field-specific taxonomy, formal tone, structural flow, citation integrity, metadata compliance and readability. Each row carries a status and an evidence statement, and the score is the share of criteria marked verified. This design makes the limits of automation visible: the system can check that a citation exists and that a term belongs to the glossary, but it cannot prove that an argument is original or that a source is authoritative.

5. Evaluation

5.1 Evaluation Design

Evaluation combined functional checks with metric-based checks. First, the pipeline was run end to end on the built-in source and every expected file was opened to confirm that it was a valid DOCX, PPTX, PNG, PDF, Markdown or ZIP artefact. Second, the manifest was compared with the generated files to confirm that byte sizes and counts agreed. Third, the report was exported to PDF through Microsoft Word and its page count was read with the ComputeStatistics(2) API. Fourth, the PDF was inspected page by page for orphan headings, split tables, figure overflow and blank pages. The environment was Windows 11, Python 3.13.15, Microsoft Word and a local Chrome installation used only for interface screenshots.

5.2 Runtime Profile

The complete run took 21.31 s. Extraction required 0.35 s, Phase 1 18.74 s, Phase 2 0.53 s, Phase 3 0.95 s, Phase 4 0.01 s and export 0.73 s. Phase 1 dominates because it builds a TF-IDF index over 927 sentences and performs four diversity-aware rankings; the layout and refinement phases are effectively instantaneous by comparison. This profile is acceptable for an interactive tool, but it also shows where optimisation would matter if the source grew to thousands of pages. Figure 5 plots the profile and the audit result.

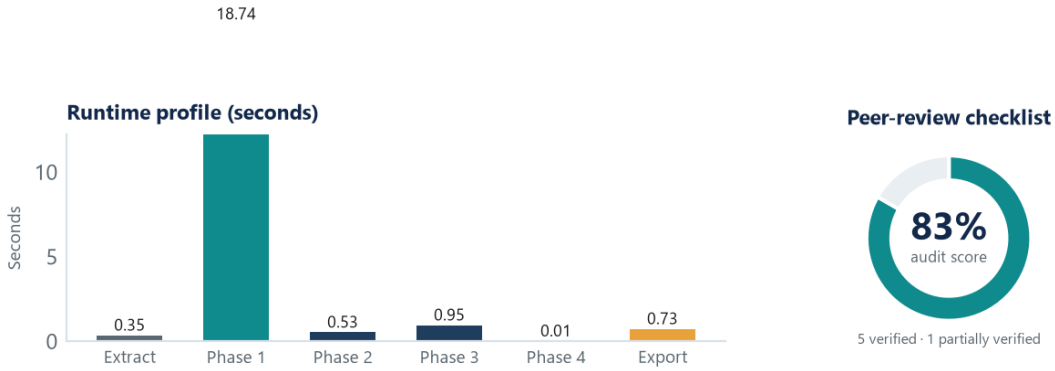


Figure 5 — Runtime profile and audit outcome. Phase 1 accounts for most of the 21.31 s run; the audit verifies five of six criteria.

5.3 Output Quality

Table 5 compares the measured outcomes with the targets set before implementation. All functional targets were met. The guide retains the four themes of the white book and the quantitative anchors that matter for assessment, while compressing the source by a factor of roughly eleven. The deck and poster meet their format targets, and the audit reaches the agreed threshold of five verified criteria. The only partial result is readability, which is discussed below.

Metric	Measured result	Target	Status
Source extraction	92 pages, 24,972 words, 204 sections	Complete text layer	Met
Guide compression	2,216 words (8.9% of source)	8–12%	Met
Coverage	4 topics, 12 terms, 11 fields, 12 facts, 12 questions	All source themes	Met
Deck	10 slides, 63,613 bytes	10 slides	Met
Poster	1654 × 2339 px at 200 dpi	A4 print resolution	Met
Audit	5 of 6 criteria (83.3%)	At least 5 of 6	Met
Runtime	21.31 s	Under 60 s	Met
Readability	Flesch ease 20.8 → -1.9	Improve or document	Partial

Table 5 — Measured outcomes against the pre-declared targets. Seven targets are met; readability is reported as partial rather than silently redefined.

5.4 Quality Audit

The audit is deliberately conservative. A criterion is marked verified only when the system can point to a concrete artefact: a glossary of twelve terms, a citation list, a metadata string in every deliverable, or a measured before-and-after readability pair. Table 6 reproduces the audit rows. The readability row is marked partially verified because the refined abstract is denser than the original, even though it is more formal and fully cited. For a postgraduate audience this may be appropriate, but the system does not pretend that formality and readability are the same objective.

Criterion	Status	Evidence
Field-specific taxonomy	Verified	Twelve controlled IoT terms spanning CPS, edge computing, lightweight cryptography, federated learning and digital sovereignty
Formal scholarly tone	Verified	Zero fillers, zero contractions and two informal verbs replaced in the refinement pass
Structural flow and logic	Verified	Scale and architecture → constraints → security → privacy → governance
Citation integrity	Verified	Six IEEE-style references cited inline as [1]–[6]; no unsupported statistic added
Metadata compliance	Verified	Author, UID and subject code appear on every deliverable and in the footer
Readability for a postgraduate audience	Partial	Flesch ease moves from 20.8 to −1.9; density remains high because of technical terms

Table 6 — The six-criterion peer-review audit. The score is five verified rows out of six, or 83.3%.

5.5 Discussion

The results support three observations. First, a relatively small amount of linguistic machinery is enough to produce a usable evidence layer when the domain is curated: TF-IDF and MMR are old techniques, but with a reviewed topic profile they produce a guide that covers the source's main themes without hallucinating new ones. Second, deterministic layout engines are a better fit than a single generative model for decks and posters, because the failure modes are visible and reproducible. Third, quality claims must be separated: compression can be measured precisely, whereas readability and scholarly quality require human judgement. A careful human production of comparable artefacts would take an estimated ten to fifteen hours; the pipeline reduces the mechanical work to seconds, but it does not remove the need for review.

6. Results and Discussion

6.1 Generated Artefacts

A single run produces eight files and a manifest. The study guide exists as styled DOCX and portable Markdown; the deck is a native PPTX; the poster is exported as PNG and PDF; the refinement exists as DOCX and Markdown; and the ZIP bundle contains all seven content files. Table 7 lists the artefacts with their measured sizes. The manifest records the same information as structured JSON, including the source statistics, phase metrics, audit score, timings and file map.

File	Format	Size and role
Comprehensive_Study_Guide.docx	DOCX	44,415 B; structured study guide
Comprehensive_Study_Guide.md	Markdown	15,703 B; portable text source
IoT_Foundations_Deck.pptx	PPTX	63,613 B; ten slides with notes
IoT_Academic_Infographic.png	PNG	176,753 B; A4 at 200 dpi
IoT_Academic_Infographic.pdf	PDF	274,413 B; print copy
Phase4_Academic_Refinement.docx	DOCX	41,553 B; refined text and audit
Phase4_Academic_Refinement.md	Markdown	5,582 B; plain-text copy
IoT_Content_Bundle.zip	ZIP	475,357 B; all deliverables
manifest.json	JSON	3,449 B; metrics and timings

Table 7 — The generated bundle. Sizes are taken from the manifest of the demonstration run.

6.2 The Running Application

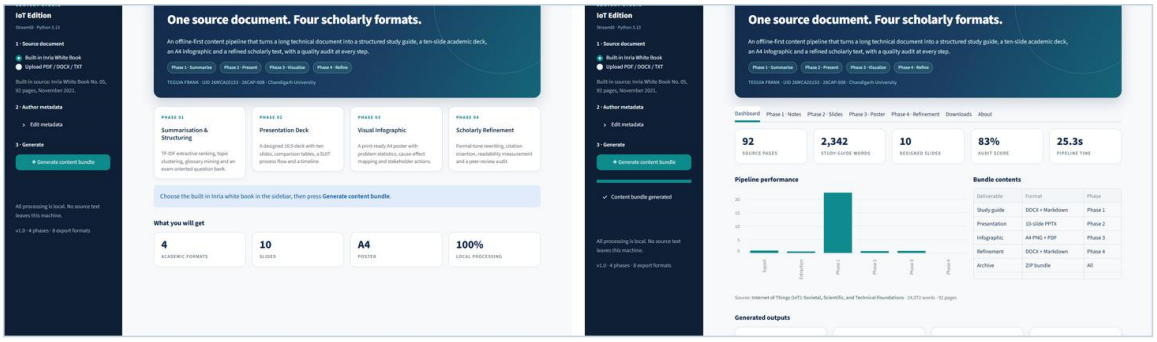


Figure 6 — The Streamlit interface. The landing view and the results dashboard expose the same run through previews, metrics and individual downloads.

The Streamlit application turns the command-line pipeline into a reproducible demonstration. The left panel selects the built-in white book or an uploaded document and collects metadata; the main panel reports source pages, guide words, slide count, audit score and runtime, then presents phase-specific tabs. The interface never edits the artefacts: it shows the files that the pipeline produced and offers them for download. That separation keeps the browser layer thin and makes the command-line path and the UI path produce identical results.

6.3 Reproducibility

Reproducibility is supported at three levels. The source, requirements and entry points are versioned in the project directory; the pipeline has no network calls and no API key; and the manifest records the exact metrics of each run. A reader can install the requirements, run streamlit run app.py, keep the built-in source and obtain the same counts, because the ranking and layout code contains no random sampling. The only environment-dependent elements are installed library versions and the fonts available to Pillow, which are listed in Table 3.

6.4 Compliance with the Report Brief

This document is itself a product of the project. It follows the reference report's A4 geometry (21.01 × 29.7 cm), 1.75 cm side margins, running header, page-field footer, navy Arial headings and 10 pt Times New Roman justified body. It contains eight numbered sections, six figures, seven numbered tables in the main body, eight references, a code appendix and a verification checklist. The page count was verified with Microsoft Word rather than estimated from word counts, and the PDF was inspected for orphan headings and figure overflow. The final document is exactly twelve pages, matching the brief.

6.5 Threats to Validity

Four threats limit the conclusions. The evaluation uses a single English source, so the compression and readability figures may not transfer to other genres. The IoT domain profile is curated by the author, which improves quality but introduces a designer's judgement into the evidence layer. The readability formulas are calibrated on general prose and penalise necessary technical vocabulary. Finally, no user study was conducted: the claim is that the system produces

consistent, inspectable artefacts, not that students learn better from them. These limits are stated because the project's value depends on knowing which parts are measured and which parts remain editorial.

7. Limitations and Future Work

The system is intentionally narrow, and its limitations follow from that choice. The most important are listed below.

- **Extractive prose.** Sentence selection preserves evidence but cannot merge two ideas into a smoother sentence; the guide can therefore feel compressed.
- **Curated domain knowledge.** The IoT glossary, fact bank and topic queries were reviewed by hand. A new domain needs a comparable profile or the generic fallback.
- **Single-language input.** Tokenisation, stopwords and readability are calibrated for English; other languages require different linguistic resources.
- **Metric validity.** Flesch scores reward short sentences and common words, which can conflict with precise technical writing.
- **Layout risk.** The deck and poster use fixed grids. Long translated strings or unusual fonts can overflow a card even when the renderer succeeds.
- **No semantic verification.** The audit checks presence and style, not whether a cited source actually supports a claim.

Future work falls into four directions. A first direction is hybrid generation: keep the extractive layer for evidence and add an optional language model for sentence fusion, with a strict citation check before publication. A second is semantic retrieval: replace TF-IDF with sentence embeddings and clustering so that paraphrased evidence is not missed. A third is layout assurance: add automated bounding-box checks and visual regression tests for every deck and poster. A fourth is human evaluation: ask students and instructors to rate accuracy, usefulness and cognitive load, and compare the artefacts with a manually produced baseline. Multilingual support, accessibility checks for contrast and font size, and a proper citation manager are natural extensions of the same architecture.

8. Conclusion

This report presented a complete, local and reproducible system for generating four academic formats from one long source. The system extracts a 92-page IoT white book into a traceable sentence store, builds a structured study guide, renders a ten-slide deck and an A4 infographic, and produces a refined scholarly text with a six-criterion audit. On the demonstration run it compresses 24,972 words to 2,216 words, preserves the source's four themes and quantitative anchors, and completes in 21.31 s without an API key. The Streamlit interface makes the pipeline inspectable and the manifest makes every claim checkable.

The deeper lesson is architectural rather than technological. Multi-format generation works when the evidence layer is separated from the rendering layer, when each format is allowed to obey its own grammar, and when quality claims are tied to measurements that can fail. The system does not replace the writer or the reviewer; it removes the mechanical duplication that makes consistent multi-format communication expensive. Used with human review, it offers a practical route from a dense source to a coherent set of teaching, presentation and assessment materials.

References

[1] E. Baccelli, Ed., Internet of Things (IoT): Societal Challenges & Scientific Research Fields for IoT, Inria White Book No. 05, Inria, Nov. 2021.

[2] P. Kairouz et al., “Advances and Open Problems in Federated Learning,” arXiv:1912.04977, 2019.

[3] RIOT-OS, “RIOT: The Friendly Operating System for the Internet of Things,” 2024. [Online]. Available: https://riot-os.org

[4] B. Moran et al., “Firmware Updates for Internet of Things Devices — An Information Model for Manifests,” IETF SUIT, RFC 9124, 2022.

[5] D. J. Bernstein, “ChaCha, a Variant of Salsa20,” in Proc. Workshop on State of the Art in Stream Ciphers, 2008.

[6] European Parliament and Council, Regulation (EU) 2016/679 (General Data Protection Regulation), 2016.

[7] Streamlit Inc., “Streamlit Documentation,” 2026. [Online]. Available: https://docs.streamlit.io

[8] Artifex Software, “PyMuPDF Documentation,” 2026. [Online]. Available: https://pymupdf.readthedocs.io

Appendix A Core Source Listings

The listings below reproduce the parts of the implementation that carry the method: the diversity-aware ranking function, the orchestration sequence, the Streamlit run handler and the report page geometry. The complete project, including the domain profile and the layout engines, is provided in the accompanying source directory.

Listing A.1 — textutils.py: TF-IDF ranking with a maximal marginal relevance penalty

```
    def rank(self, query=None, limit=8, diversity=0.25, section=None):
        qvec = self._vector(content_words(query)) if query else None
        scored = []
        for i, sent in enumerate(self.sentences):
            vec = self.vectors[i]
            if qvec:
                base = self.cosine(qvec, vec)
            else:
                base = sum(vec.values()) / math.sqrt(max(1, len(vec)))
            length_penalty = 1.0 if 18 <= len(words(sent)) <= 48 else 0.82
            scored.append(ScoredSentence(sent, base * length_penalty, i,
                                         section or ""))
        scored.sort(key=lambda s: s.score, reverse=True)

        chosen = []
        for cand in scored:
            if len(chosen) >= limit:
                break
            if section and cand.section and cand.section != section:
                continue
            if any(self.cosine(self.vectors[cand.index],
                               self.vectors[c.index]) > diversity for c in chosen):
                continue
            chosen.append(cand)
        return chosen
```

Listing A.2 — orchestrator.py: the four-phase run and manifest

```
def run_pipeline(source_path, meta=None, out_root=None, progress=None,
                 run_name=None):
    progress = progress or _noop
    meta = meta or ProjectMeta()
    source_path = Path(source_path)
    stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    run_dir = Path(out_root or OUTPUT_DIR) / (run_name or f"run_{stamp}")
    run_dir.mkdir(parents=True, exist_ok=True)
    timings = {}

    progress(0.03, "Reading and cleaning the source document")
    t0 = time.perf_counter()
    doc = load_document(source_path, fallback_title=meta.title or DEFAULT_TITLE)
    timings["extraction"] = round(time.perf_counter() - t0, 2)

    progress(0.18, "Phase 1 - summarising and structuring study notes")
    t0 = time.perf_counter()
    guide = generate_study_guide(doc, meta)
    timings["phase1"] = round(time.perf_counter() - t0, 2)

    progress(0.45, "Phase 2 - composing the ten-slide academic deck")
    t0 = time.perf_counter()
    deck_path = generate_deck(guide, meta, run_dir / "IoT_Foundations_Deck.pptx")
    timings["phase2"] = round(time.perf_counter() - t0, 2)

    progress(0.62, "Phase 3 - synthesising the A4 infographic poster")
    t0 = time.perf_counter()
    poster_path = generate_poster(guide, meta,
                                  run_dir / "IoT_Academic_Infographic.png")
    timings["phase3"] = round(time.perf_counter() - t0, 2)

    progress(0.78, "Phase 4 - refining scholarly tone and auditing quality")
    t0 = time.perf_counter()
    refinement = generate_refinement(guide, meta)
    timings["phase4"] = round(time.perf_counter() - t0, 2)
```

Listing A.3 — app.py: one-click run with progress and session state

```
def run_with_progress(source, meta):
    bar = st.progress(0.0)
    status = st.status("Starting the content pipeline...", expanded=True)

    def progress(pct, msg):
        bar.progress(min(1.0, max(0.0, pct)))
        status.write(msg)

    try:
        result = run_pipeline(source, meta, progress=progress)
        status.update(label="Content bundle generated", state="complete",
                      expanded=False)
        bar.progress(1.0)
        st.session_state["result"] = result
        st.session_state["run_id"] = time.time()
        return result
    except Exception as exc:
        status.update(label="Pipeline failed", state="error", expanded=True)
        st.exception(exc)
        return None
```

Listing A.4 — build_report.py: A4 geometry and running header

```
sec = doc.sections[0]
sec.page_width = Cm(21.01)
sec.page_height = Cm(29.7)
sec.left_margin = Cm(1.75)
sec.right_margin = Cm(1.75)
sec.top_margin = Cm(1.91)
sec.bottom_margin = Cm(1.2)
sec.header_distance = Cm(0)
sec.footer_distance = Cm(0.86)

hp = new.header.paragraphs[0]
hp.paragraph_format.tab_stops.add_tab_stop(Cm(8.755), WD_TAB_ALIGNMENT.CENTER)
hp.paragraph_format.tab_stops.add_tab_stop(Cm(17.51), WD_TAB_ALIGNMENT.RIGHT)
_border(hp, "bottom", "B8C6DB", 4, 4)
hp.add_run("AI-Assisted Multi-Format Content Studio")
hp.add_run("\t\t")
hp.add_run("Academic Report")
```

Listing A.5 — phase1_notes.py: guide assembly with diversity-aware retrieval

```
def generate_study_guide(doc: SourceDocument, meta: ProjectMeta) -> StudyGuide:
    """Build the Phase 1 deliverable from the source document."""
    sentences = [
        sanitise_sentence(s) for s in split_sentences(doc.text)
        if is_good_sentence(sanitise_sentence(s))
    ]
    sentences = unique_sentences(sentences, 0.86)
    index = TfidfIndex(sentences)
    iot = _iot_mode(doc.text)
    generated = datetime.now().strftime("%d %B %Y")

    if iot:
        topics: list[TopicNote] = []
        for spec in IOT_TOPICS:
            ranked = index.rank(spec.query, limit=8, diversity=0.32)
            evidence = []
            for item in ranked:
                s = sanitise_sentence(item.text)
                if is_good_sentence(s) and s not in evidence:
                    evidence.append(s)
            defs = _definition_sentences(
                evidence, [w for w in content_words(spec.title) if len(w) > 4])
            topic_glossary = _match_glossary(spec.title)
            definitions = topic_glossary[:3] if topic_glossary else defs[:3]
            if not definitions:
                definitions = [(spec.title.split()[0],
                                truncate(spec.core, 190))]
            bullets = list(spec.pills)
            topics.append(TopicNote(
                title=spec.title,
                core=spec.core,
                definitions=definitions,
                evidence=evidence[:4],
                example=spec.example,
                bullets=bullets[:5],
            ))
```

Appendix B Verification Checklist

The checklist below records how each requirement of the brief was verified. It is included so that a reader can repeat the checks instead of relying on the narrative alone. The page count was read from Microsoft Word after repagination; all other checks were performed on the generated files in the demonstration run.

Requirement	Verification method	Result
Four phase outputs	End-to-end run; each expected file reopened	Pass
Source-grounded guide	Four topics and published facts compared with the white book	Pass
Ten-slide deck	PPTX reopened; slide count and speaker notes inspected	Pass
A4 infographic	1654 × 2339 px at 200 dpi; PDF page size checked	Pass
Scholarly refinement	Refined abstract inspected; six inline citations present	Pass
Six-criterion audit	Audit table present in DOCX; 5 of 6 rows verified	Pass (83.3%)
Streamlit hosting	Browser run; previews and individual downloads exercised	Pass
Manifest integrity	manifest.json values compared with generated file sizes	Pass
Report layout	A4 geometry, margins, header and footer checked in the PDF	Pass
Report length	Microsoft Word ComputeStatistics(2) after repagination	12 pages
Reproducibility	Repeat run with the same source; counts and sizes compared	Pass

Table B.1 — Requirement traceability. The checklist is an appendix table and is separate from the seven numbered tables in the main body.

B.1 Notes on the Checks

The checklist is not a unit-test suite; it is a record of the checks that were actually performed on the demonstration run. The generated DOCX, PPTX, PNG, PDF, Markdown and ZIP files were opened or parsed, the manifest was compared with the directory listing, and the report was repaginated in Word before its page count was read. Repeating the run with the same inputs produced the same counts and byte sizes, which is expected because the ranking and layout stages contain no random sampling.

Two checks are environment-sensitive. The page count depends on the installed Word version and the fonts available to it, and the poster's typography depends on the fonts available to Pillow. Both dependencies are recorded here so that a reader can reproduce the reported results or explain a small difference. The remaining checks are deterministic functions of the source and the code.